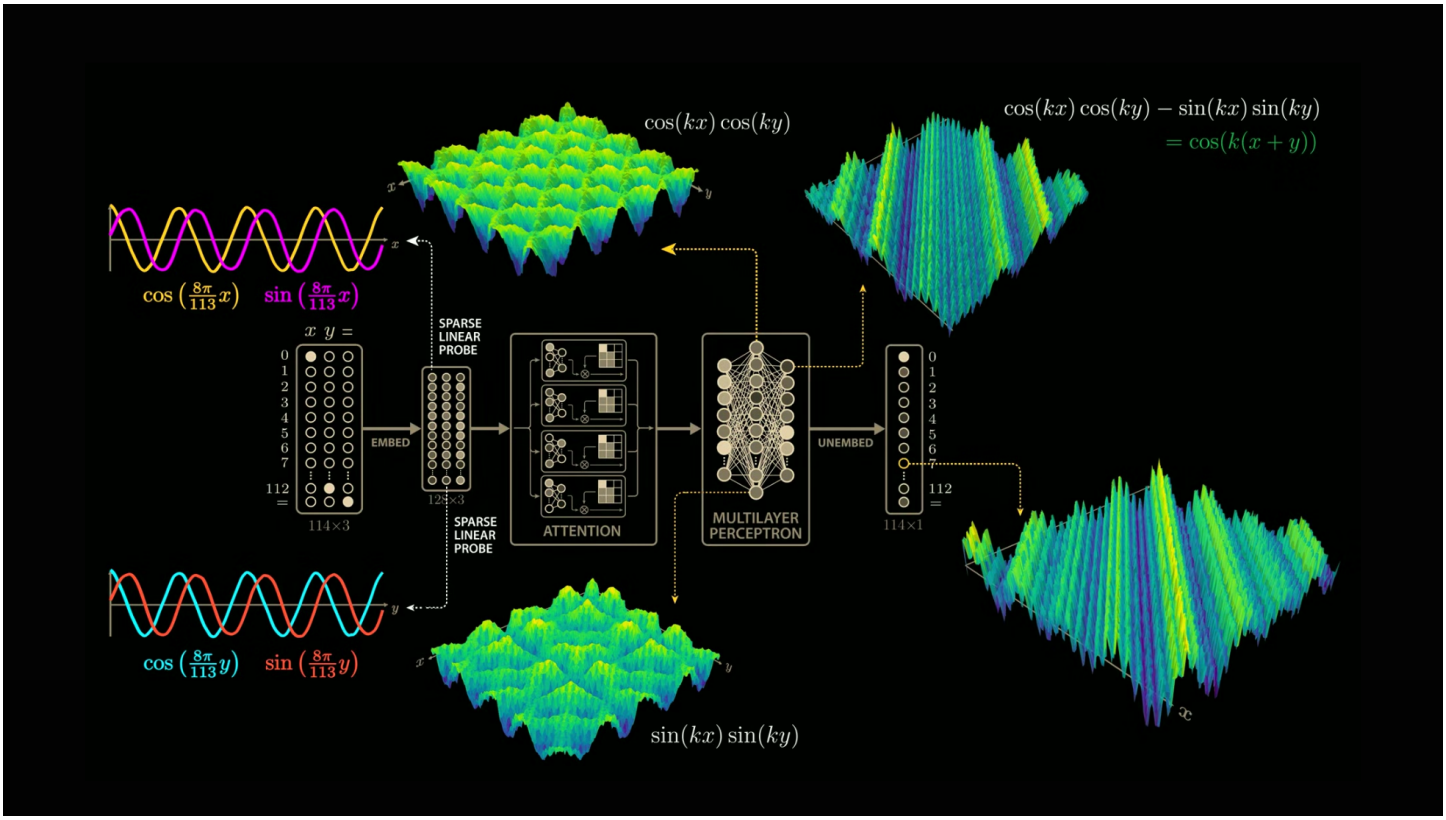
Slide 1



Let the trig identity sit on screen for a second. Let people read it and groan.

So... when was the last time you learned a trigonometric identity?

Pause.

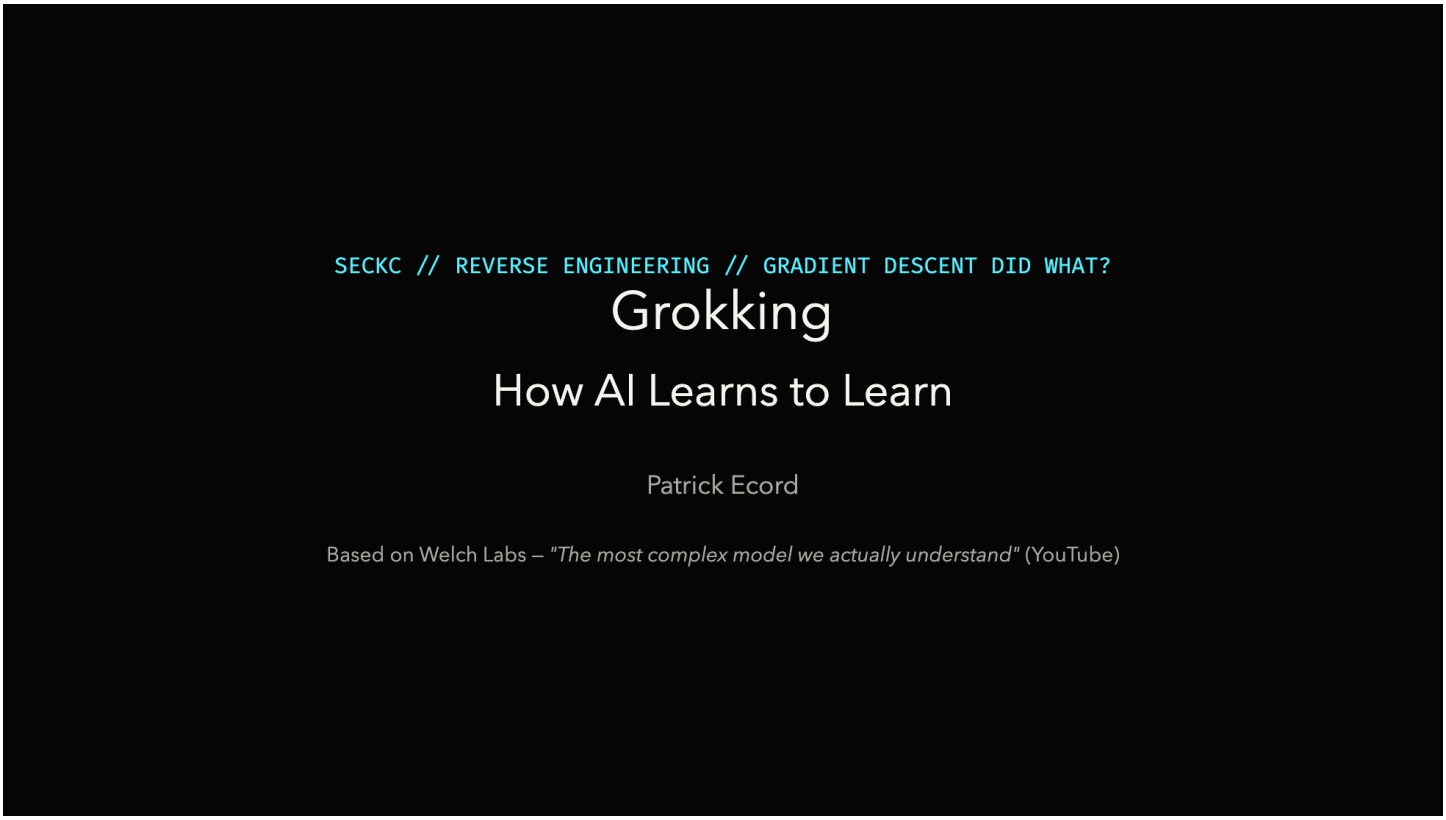
Yeah. Same.

Beat.

Yeah well neither did this clanker.

Tonight we're going to look at how a model learns math. Kinda sorta.

Slide 2 — How AI Learns to Learn

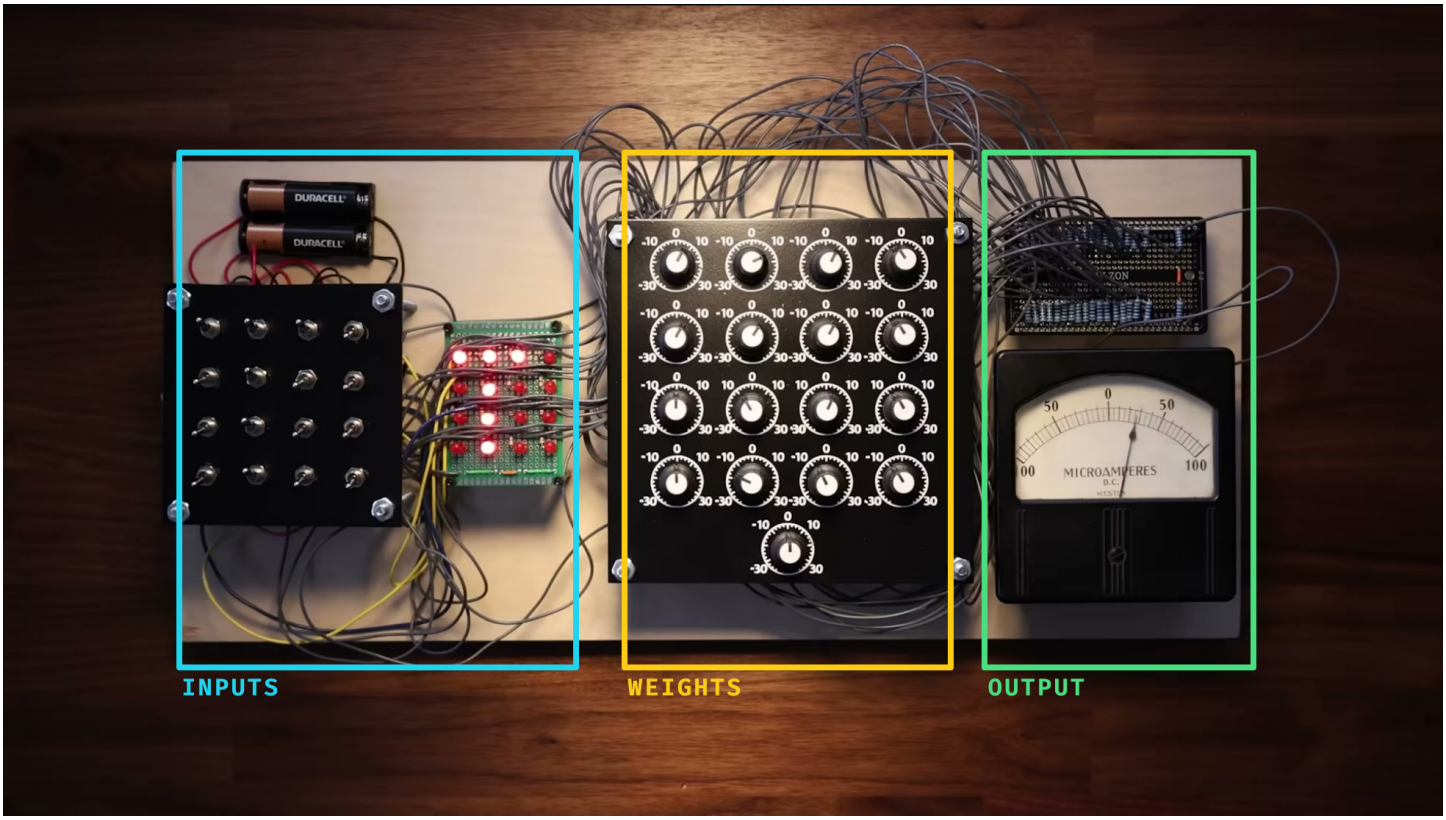


Hello everyone. My name is Patrick. It's great to see your beautiful faces.

Tonight we're going to talk about how AI learns to learn. We're going under the hood — we'll see what the model sees, and how training works, how a pattern emerges from noise.

And hopefully by the end of tonight, we'll have a better idea of how it works at a base level.

Slide 3 — How AI Learns to Learn



This is an artificial neuron, also called a perceptron, built as a physical machine in the 1950s.

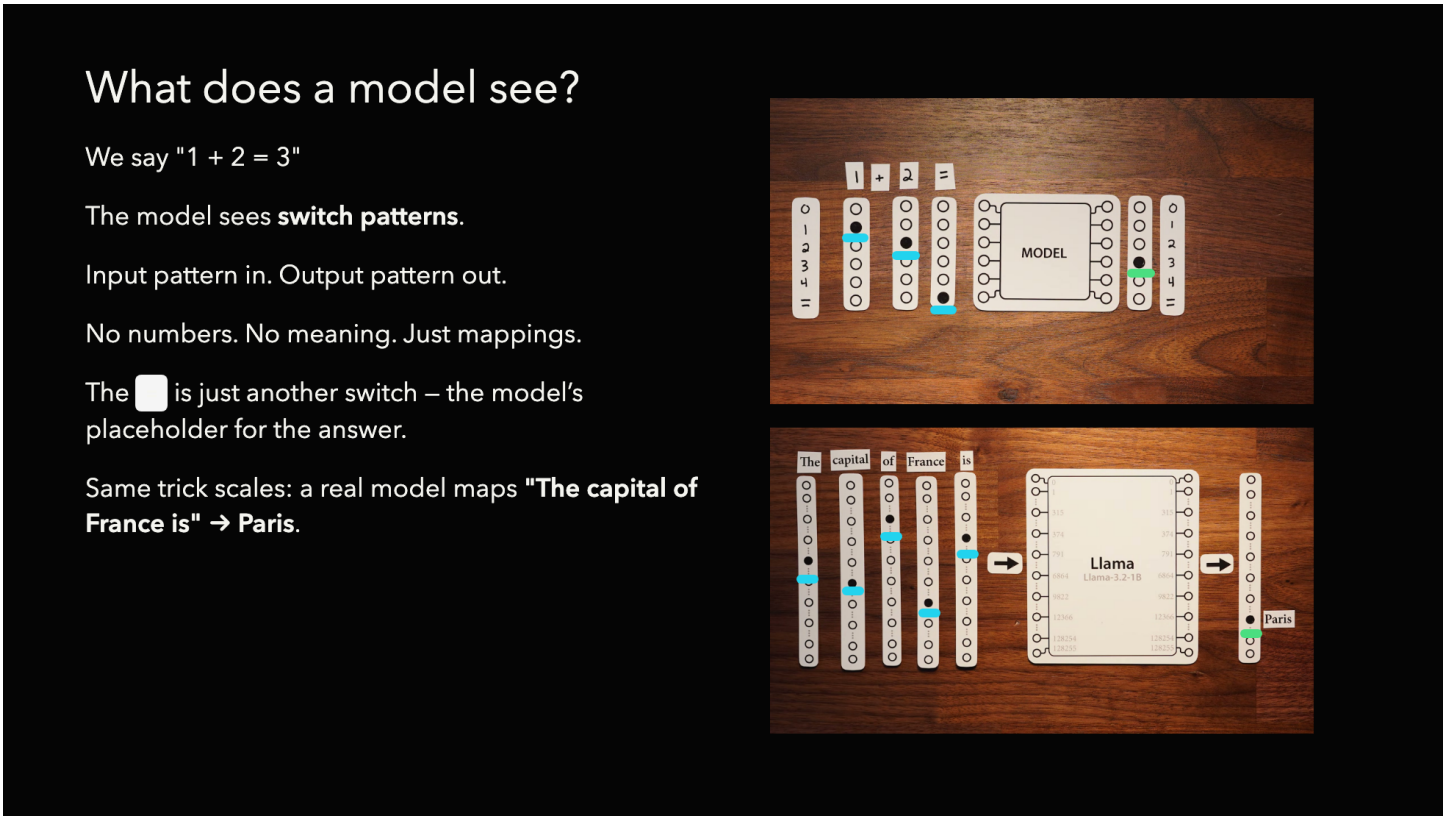
The switches on the left are the inputs, the dials are the weights, the meter on the right is the output.

Right now the inputs are set to a T — you can see that T lit up, those are turned on, everything else off. The dials in the middle are tuned to detect that T, so when the neuron sees it, the meter on the right swings positive.

Modern models have billions or trillions of these, but the idea is basically the same: you feed in a pattern, you adjust the weights, and you get the output you want.

We're going to start with this simple machine and reverse engineer how a tiny model learned modulus addition.

Slide 4 — What does a model see?



So here's an example: 1 + 2 = 3. Pretty straightforward, right? (at least to some of us)

But the model doesn't know what a number is, or an equals sign — it's us humans who give these symbols meaning. Imagine the numbers covered up: all the model sees is a switch flipped on for "1", then "2", then "=", and it learns that when it sees that pattern, it outputs a 3.

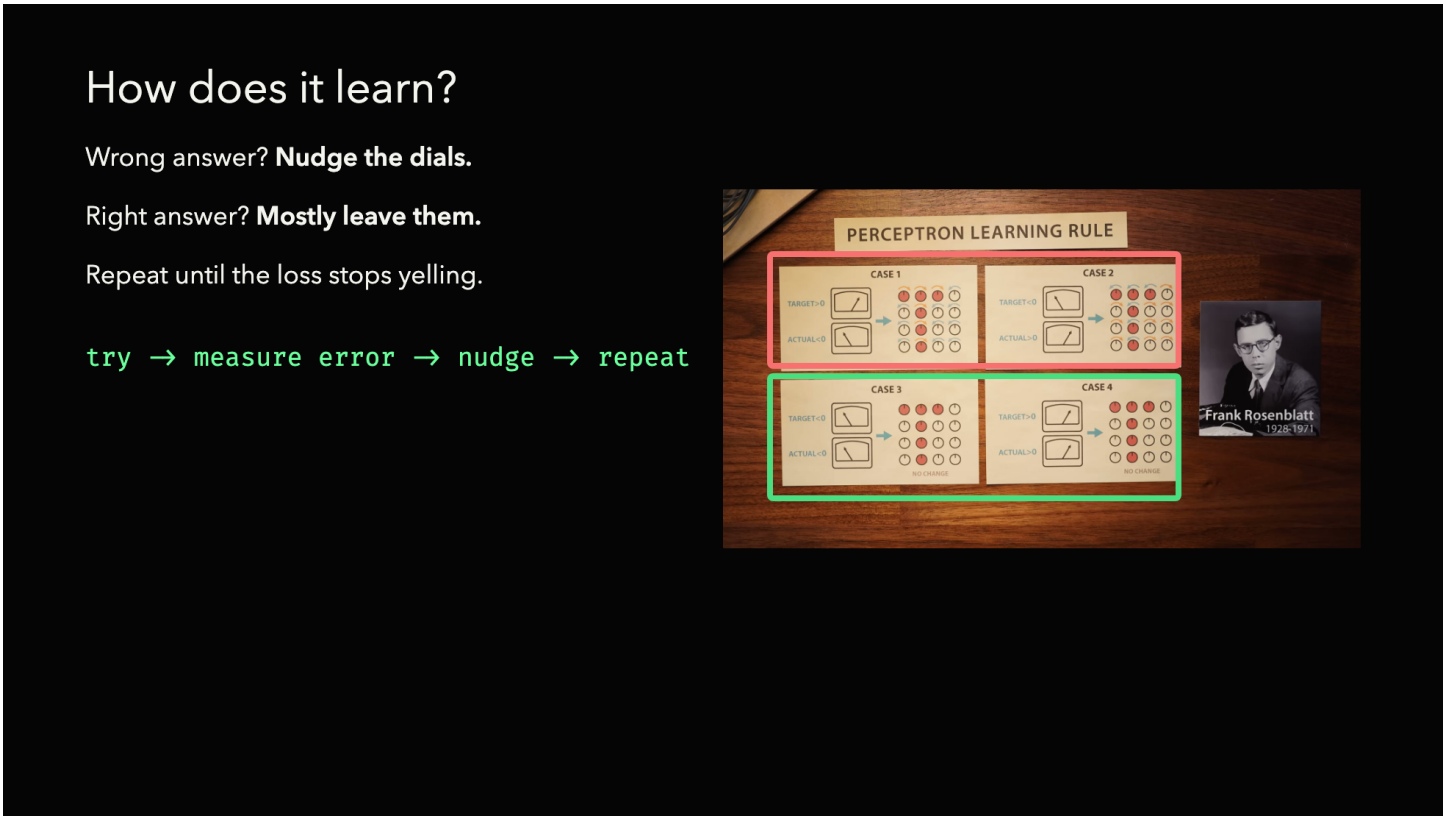
You might be wondering why we have an equals symbol but no addition symbol — the equals sign gives the model a placeholder for where the answer goes. It's kind of like the end of a sentence.

And the same thing happens with a real LLM. That second screenshot is Llama 3.2 (1B), with a 128k vocab — plug in "the capital of France is" and those input tokens light up, and it outputs Paris.

But to the model, this is pattern in, pattern out. That's the whole game.

So if we want to know whether it learned the rule, we need to inspect what happens inside while the dials are being turned.

Slide 5 — How does it learn?



The way it learns isn't magic, it's a feedback loop.

You try an answer, measure how wrong it was, nudge the dials in the right direction, and do that an absurd number of times.

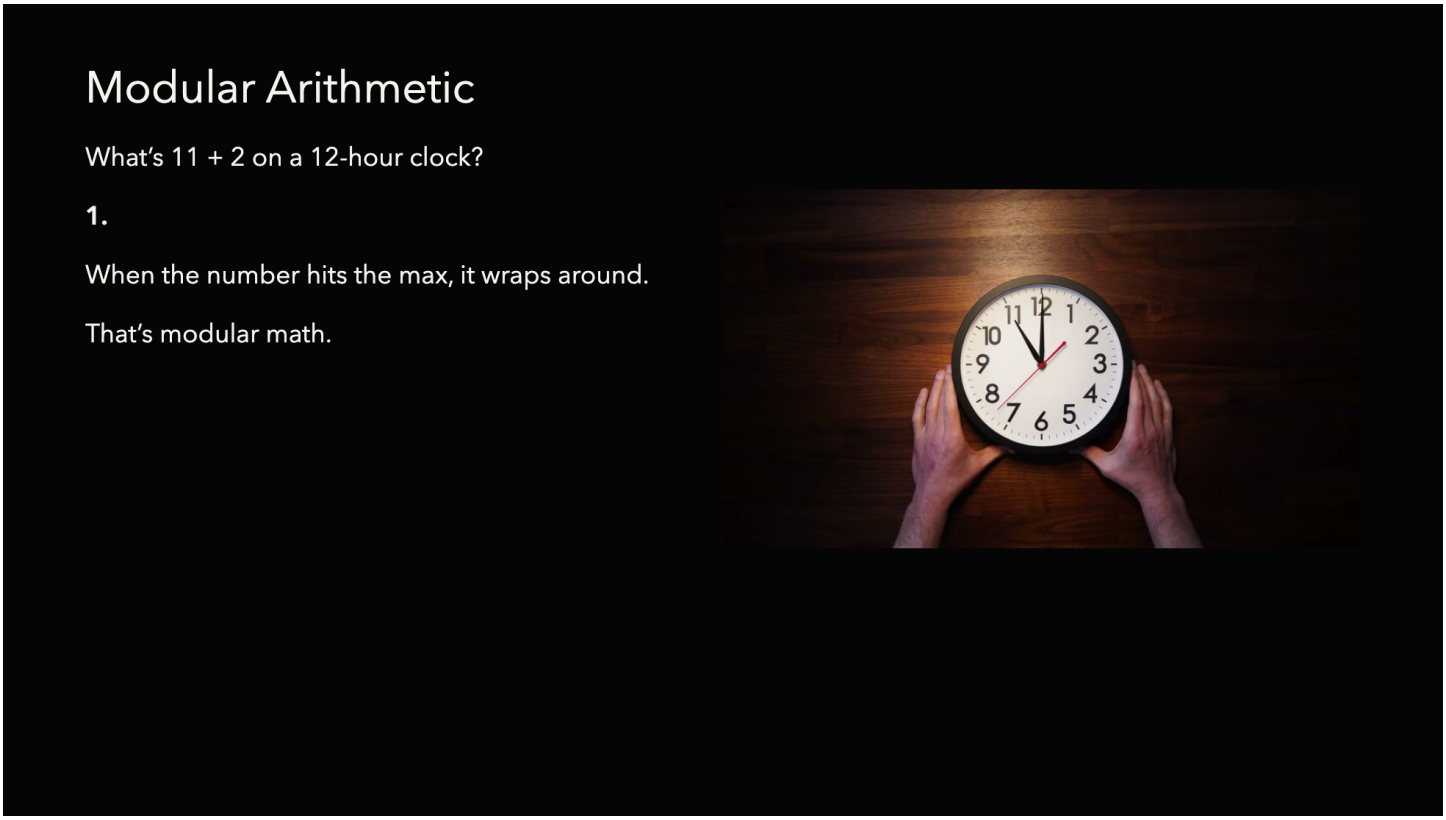
Here's the actual rule — the four cases on screen. If the target should be positive but the output came out negative, you turn the weights up. The other way around — target negative, output positive — you turn them down. And if the output already matches the target, you leave it alone. No change.

This is basically backpropagation. Think back to the capital of France: early in training the model guesses wrong, you show it the right answer, it measures how far off it was, and it nudges its weights to do better next time.

A real model has way more weights than you could ever turn by hand, but the mental model is the same — training nudges the weights until the output looks right.

And this rule is older than you'd think — Frank Rosenblatt came up with the perceptron back in the 1950s.

Slide 6 — Modular Arithmetic

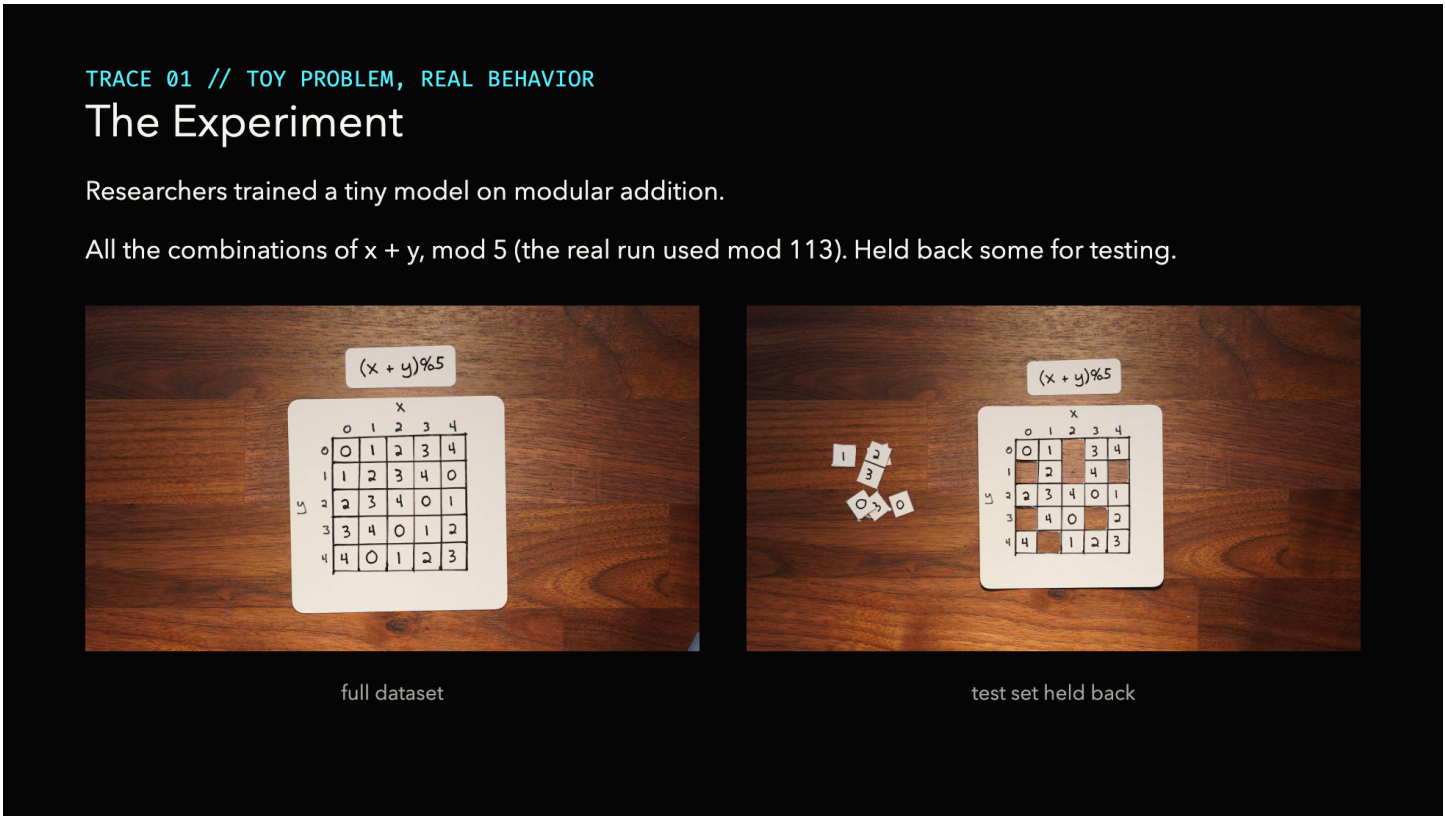


Before I show you the weird thing, I need to explain one concept: modular arithmetic.

Don't let the name scare you — you already know this. What's 11 + 2 on a 12-hour clock? It's 1. Not 13 — 1. Because when you hit 12, the clock wraps back around to 1. (unless you are using 24 hour time like a nerd)

That wraparound is the whole trick. Keep the clock in your head, because the model is going to independently discover a version of it.

Slide 7 — The Experiment



Researchers set up a simple experiment.

They took a tiny neural network — way smaller than anything you'd use in production — and trained it to do modular addition. Specifically, A plus B mod 113.

This is a small example showing every combination of x and y, mod 5. On the right, you can see they cut out some of the squares — that's the test set the model never sees. It trains on the ones that are left, then gets evaluated on the cut-out ones.

The model's job: figure out the pattern. Learn to do this addition.

Simple enough, right?

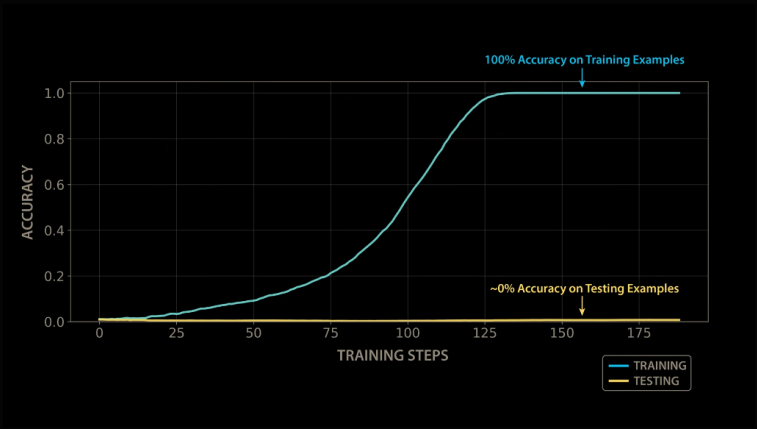
Slide 8 — The Result

TRACE 02 // LOOKS LIKE MEMORIZATION

The Result

100% accuracy on training examples – memorized the answers

~0% accuracy on testing examples – can't do new problems



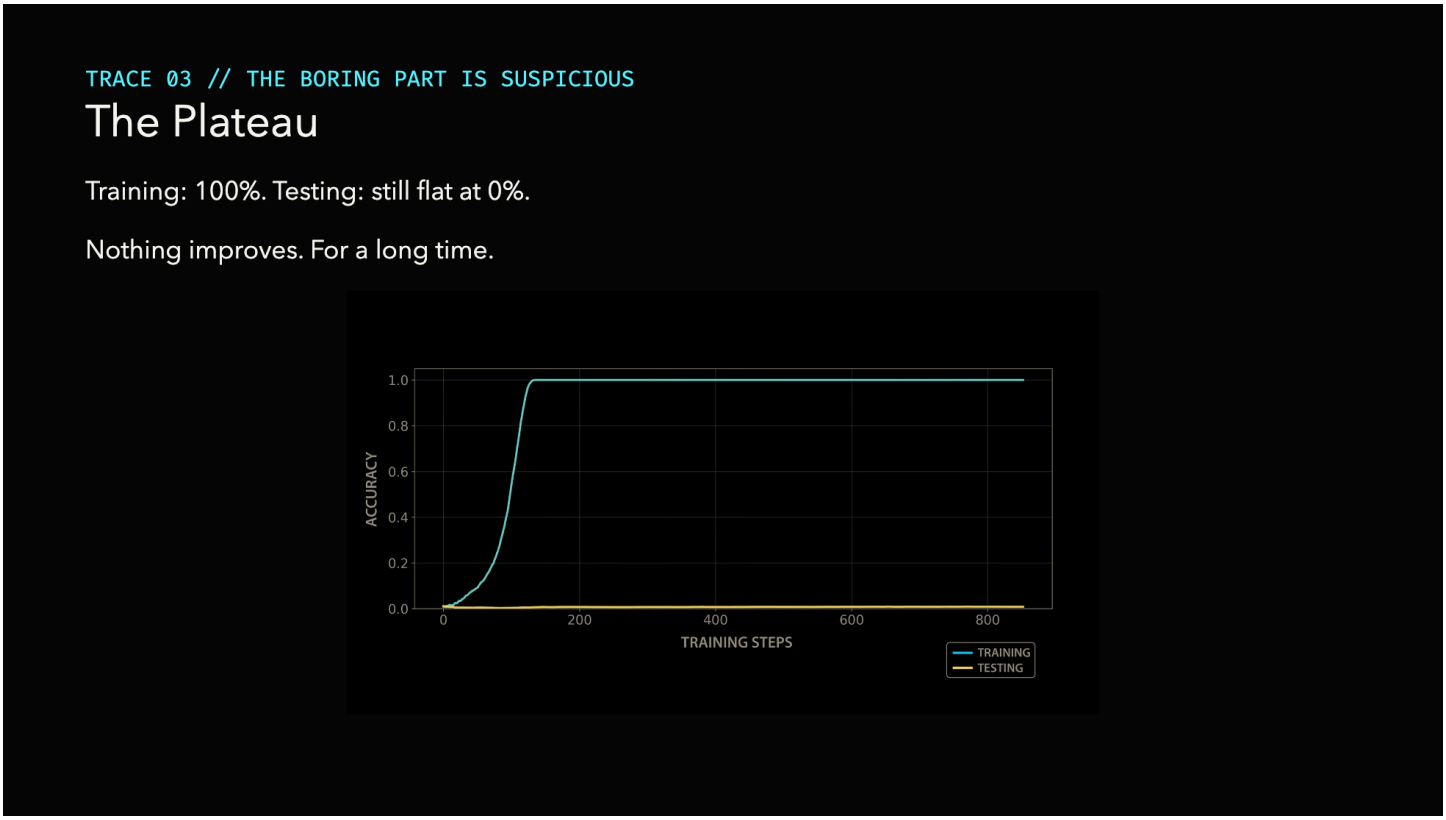
And here's what happened. The blue line is accuracy on the training data — the problems it saw during training. It shoots up to 100%. The model memorized every single answer.

But the yellow line — that's accuracy on the test set. The problems it hadn't seen. Flat. Basically zero.

It's like a student who memorized every answer on the practice exam but can't solve a single new problem. It didn't learn addition — it learned a lookup table.

If you're a machine learning engineer, you look at this and say 'classic overfitting' and you stop training. You're done. The model learned what it's going to learn.

Slide 9 — The Plateau



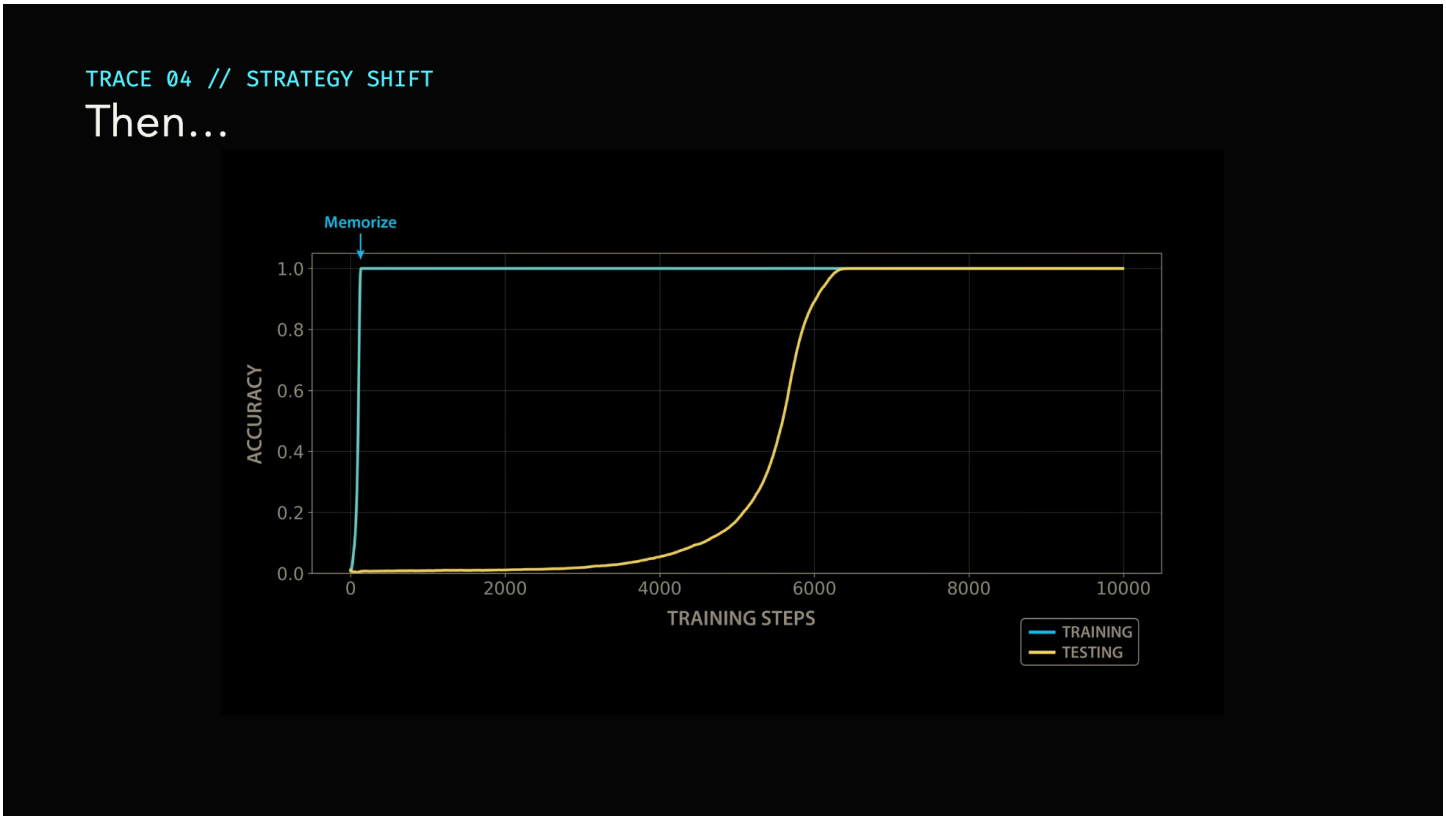
And if you keep training... nothing happens. For a long time.

The training accuracy stays at 100%. The test accuracy stays near zero. Step after step after step. Nothing changes.

You would absolutely stop training here. There's zero indication that anything useful is going to happen. It looks like wasted compute.

But the experiment keeps going. And the weird part is that the boring flat line isn't actually boring inside the model.

Slide 10 — Then...



Let the image do the work. Pause before speaking.

As the story goes, this was almost found by accident — someone left one of these models training way longer than anyone normally would. And when they came back...

...the test accuracy just... snaps to 100%.

Not gradual. Not a slow improvement. It goes from basically zero to perfect in a tiny window of training steps.

Long pause. Let it sink in.

It stopped memorizing... and started solving.

It looked like nothing was happening during training, but inside, the model was quietly building patterns. And then it clicked.

And they thought: what the hell is going on inside this model?

Slide 11 — That's grokking

That's grokking

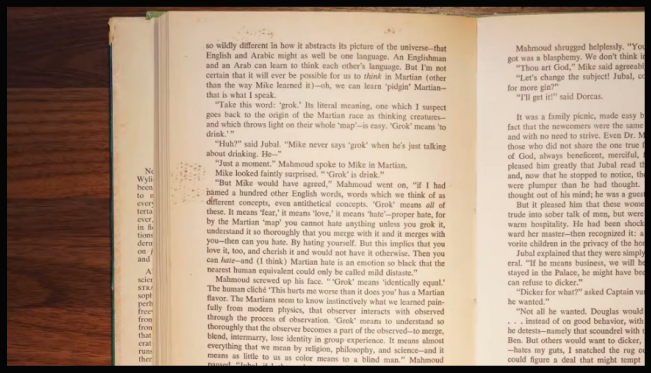
A model looks like it memorized the training set…

then suddenly generalizes to the rule.

`overfit → plateau → snap`

"You cannot hate anything unless you grok it – understand it so thoroughly that you merge with it, and it merges with you."

– Robert A. Heinlein, *Stranger in a Strange Land*


so wildly different in how it abstracts its picture of the universe–that English and Arabic might as well be one language. An Englishman and an Arab can learn to think each other's language. But I'm not certain that it will ever be possible for us to *think* in Martian (other than the way Mike learned it)–oh, we can learn 'pidgin' Martian–that is what I speak.

"Take this word: 'grok.' Its literal meaning, one which I suspect goes back to the origin of the Martian race as thinking creatures–and which throws light on their whole 'map'–is easy. 'Grok' means 'to drink.'"

"Huh?" said Jubal. "Mike never says 'grok' when he's just talking about drinking. He–"

"Just a moment." Mahmoud spoke to Mike in Martian.

Mike looked faintly surprised. "'Grok' is drink."

"But Mike would have agreed," Mahmoud went on, "if I had named a hundred other English words, words which we think of as different concepts, even antithetical concepts. 'Grok' means *all* of these. It means 'fear,' it means 'love,' it means 'hate'–proper hate, for by the Martian 'map' you cannot hate anything unless you grok it, understand it so thoroughly that you merge with it and it merges with you–then can you hate. By hating yourself. But this implies that you love it, too, and cherish it and would not have it otherwise. Then you can *hate*–and (I think) Martian hate is an emotion so black that the nearest human equivalent could only be called mild distaste."

Mahmoud screwed up his face. "'Grok' means 'identically equal.' The human cliché 'This hurts me worse than it does you' has a Martian flavor. The Martians seem to know instinctively what we learned painfully from modern physics, that observer interacts with observed through the process of observation. 'Grok' means to understand so thoroughly that the observer becomes a part of the observed–to merge, blend, intermarry, lose identity in group experience. It means almost everything that we mean by religion, philosophy, and science–and it means as little to us as color means to a blind man." Mahmoud

Mahmoud shrugged helplessly. "Yo
got was a blasphemy. We don't think
"Thou art God," Mike said agreeab
"Let's change the subject! Jubal,
for more gin?"
"I'll get it!" said Dorcas.

It was a family picnic, made easy b
fact that the newcomers were the same
and with no need to strive. Even Dr. M
those who did not share the one true f
of God, always beneficent, merciful,
pleased him greatly that Jubal read th
and, now that he stopped to notice, th
were plumper than he had thought.
thought out of his mind; he was a guest
But it pleased him that these wonde
trude into sober talk of men, but were
warm hospitality. He had been shock
ward her master–then recognized it: a
vorite children in the privacy of the ho
Jubal explained that they were simply
eral. "If he means business, we will he
stayed in the Palace, he might have bee
can refuse to dicker."
"Dicker for what?" asked Captain Va
he wanted."
"Not all he wanted. Douglas would
. . . instead of on good behavior, with
he detests–namely that scoundrel with
Ben. But others would want to dicker,
–hates my guts, I snatched the rug o
could figure a deal that might tempt

This delayed snap is what the researchers called grokking.

The word comes from Robert Heinlein's Stranger in a Strange Land. To grok something is to understand it deeply, not just know the answer.

In this talk we're using it in the machine learning sense: at first the model memorizes the examples, then later finds the rule that generalizes.

The next question is the fun one: what changed inside the model?

Slide 12 — So we popped the hood.

Now we dig into the model — treat it like an unknown binary and reverse engineer it. We're going to look at the weights and activations and try to understand how it actually learned the rule.

No source code that says 'solve modular addition.' Just weights, activations, and traces.

The obvious expectation is messy lookup behavior: arbitrary internal state that happens to produce the right answers.

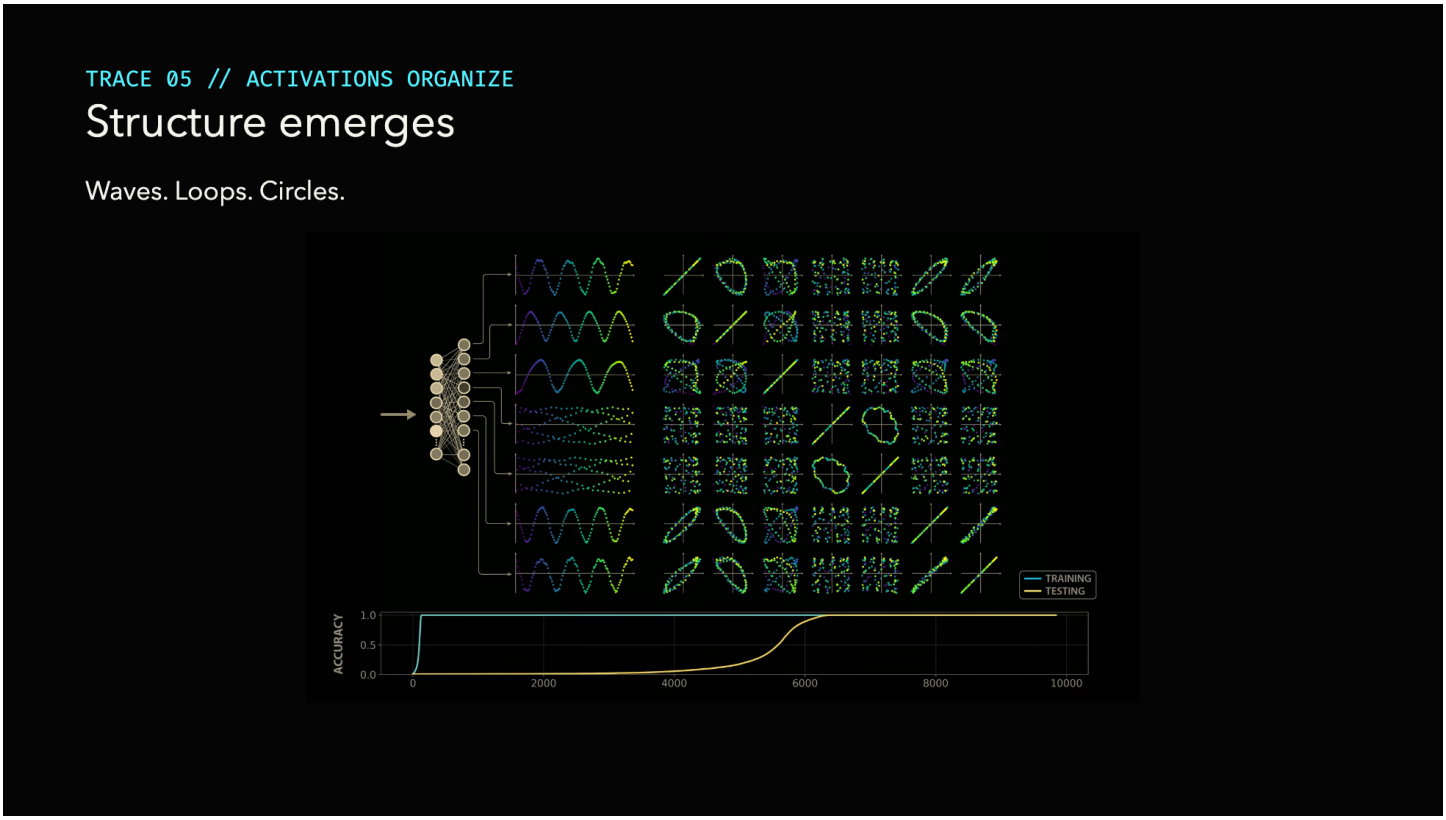
What they actually found... watch this.

Let the gif play through once. It shows structure emerging from noise.

You're watching the internal structure of the model evolve as it trains. It goes from complete noise — random scattered dots — to clean geometric patterns. Waves. Loops. Circles.

Something organized is happening inside this model. Something nobody asked it to do.

Slide 13 — Structure emerges

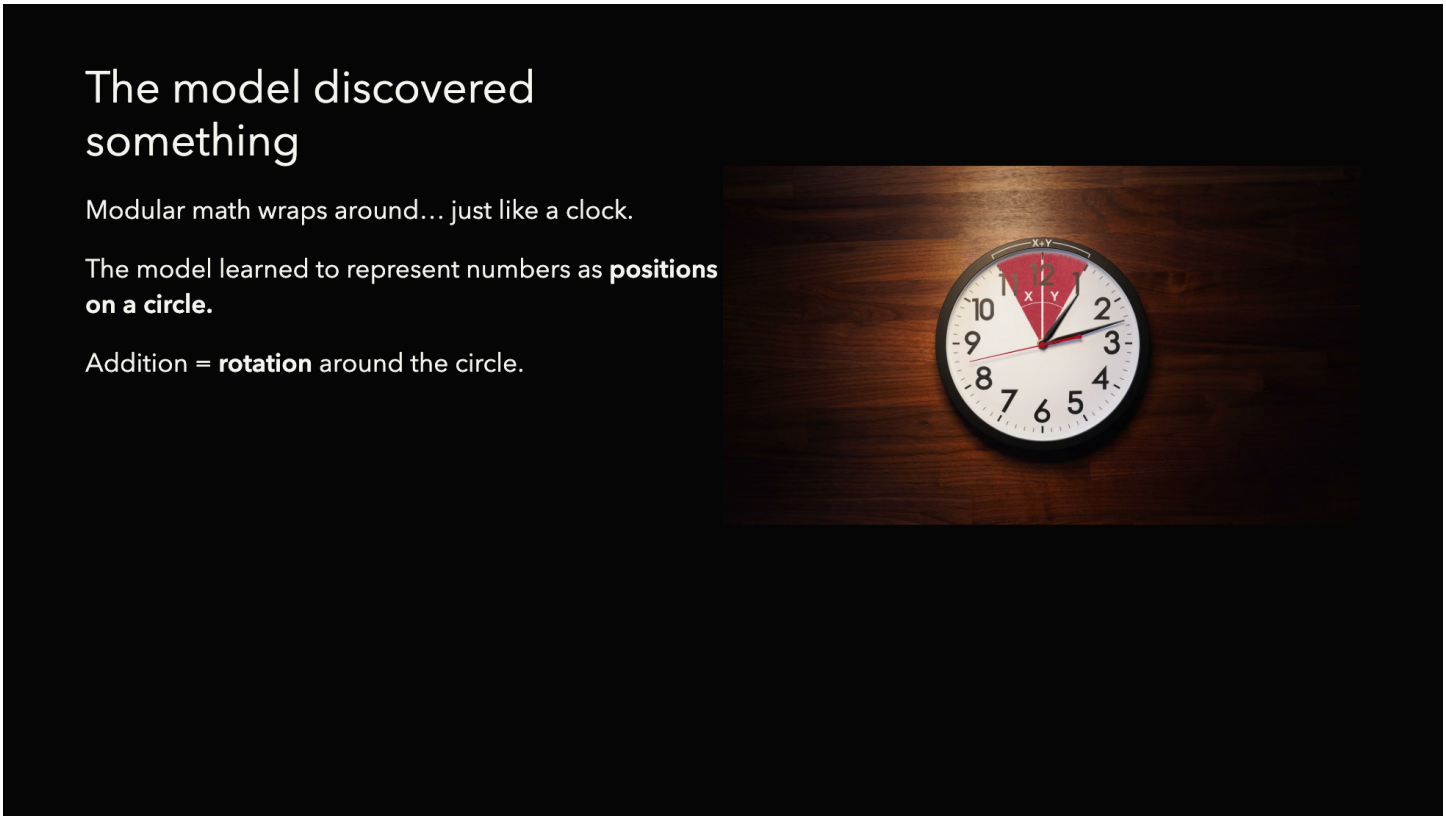


Here's what the fully trained model looks like inside. This is a single-layer transformer — the inputs come in and get transformed into these activations.

On the left are the activations themselves — clean sine waves. On the right, we plot pairs of neurons against each other. A neuron plotted against itself is just a straight line. But plot the first neuron against the second, and it curls into a loop — a circle.

This is the moment where the autopsy gets interesting. This is supposed to be a model that does addition. Why is it drawing circles?

Slide 14 — The model discovered something



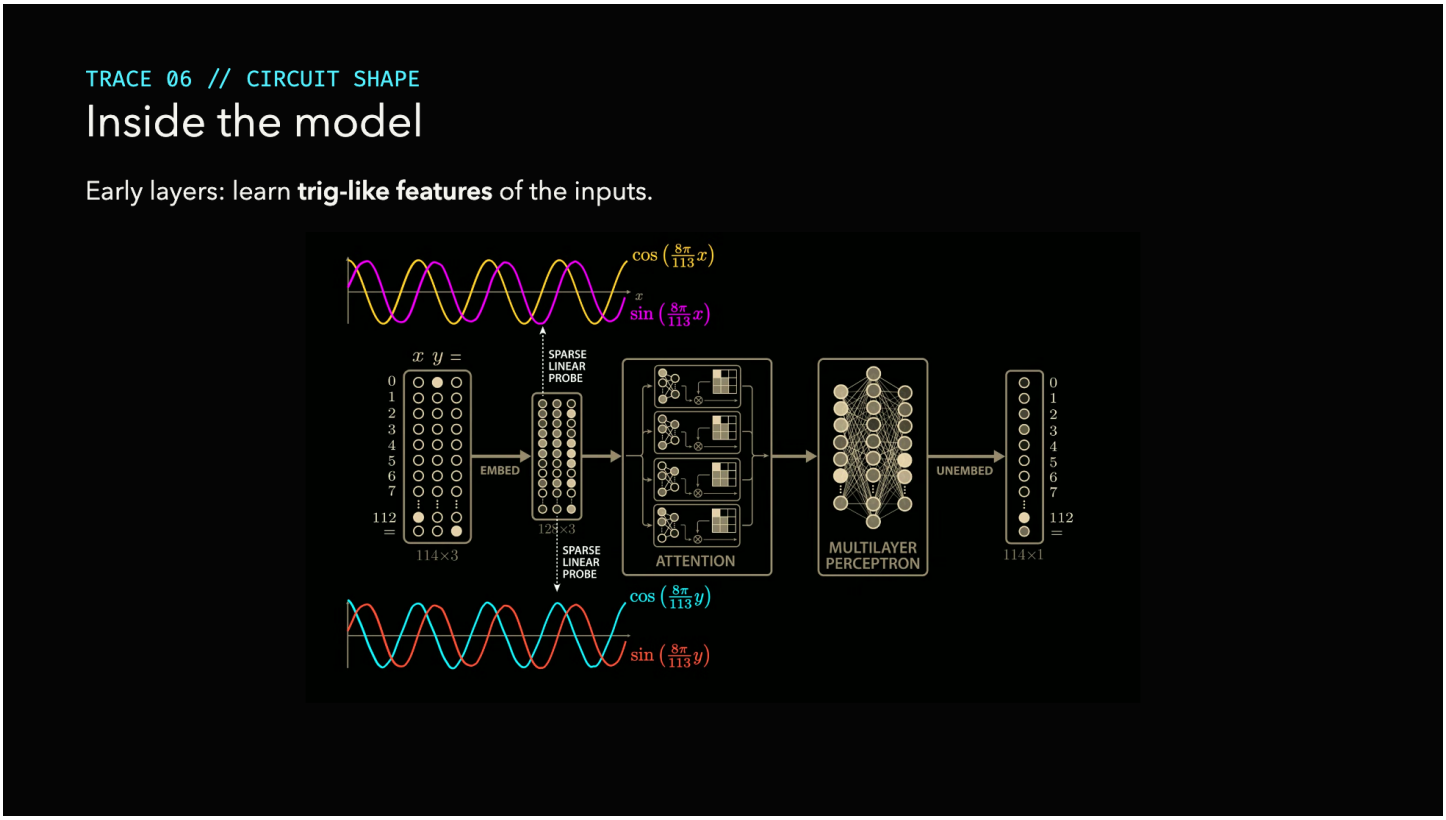
You guys remember the clock from earlier — how modular arithmetic wraps around, just like a clock face? Well, the model figured that out on its own too.

It learned to represent numbers as a position on a circle, and it discovered that that was a useful way to think about the problem.

And so addition just becomes rotation around that circle. 11 + 2? You land on 1 — you can see that in the picture here. The model's doing the same thing with its own internal representation.

The model isn't thinking in numbers, it's thinking in positions on a circle. Nobody told it this — the training data only implied it, and the model figured out it was an easy way to solve the problem.

Slide 15 — Inside the model



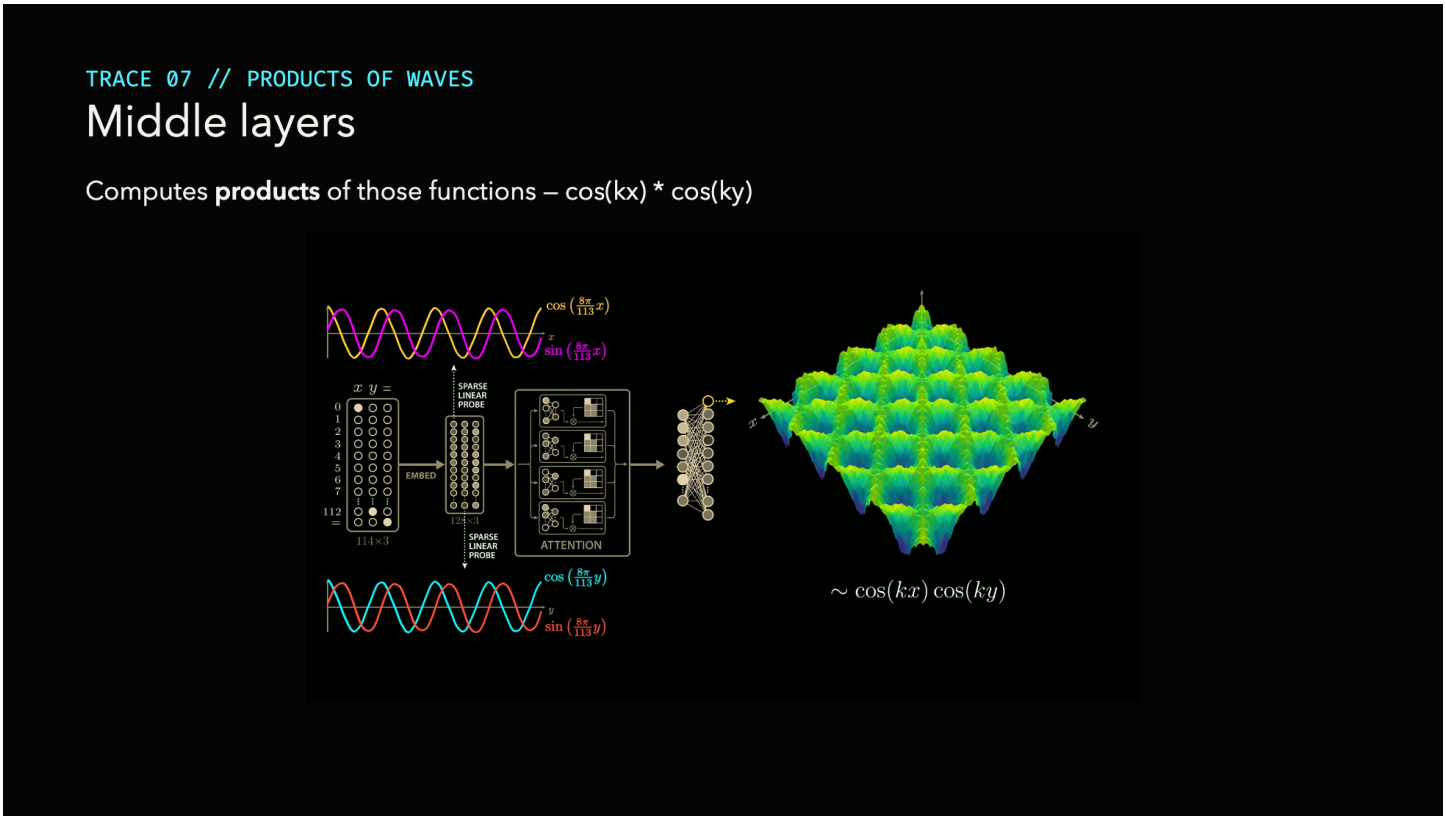
Let's get specific about what's actually happening inside the model.

Our input comes in on the left and goes into a sparse linear probe — the x column gives us cosine of x and sine of x, and the y column gives us cosine of y and sine of y. Then it feeds into the attention and the multi-layer perceptron, and out comes an answer.

You might be wondering: why is it learning sine and cosine? If you remember anything from math class — or even if you don't — sine and cosine are just the x and y coordinates of a point on a circle.

The model learned to put numbers on a circle. On its own.

Slide 16 — Middle layers



In these middle layers is where it actually gets interesting. The model starts multiplying those trig functions together.

What we're looking at is a 3D surface — the output of a single neuron as you vary both x and y. One axis is x, one is y, and the surface is the combination of them. The dominant pattern is cosine of x times cosine of y.

It probably seems random — why would multiplying cosines together help you add? Stick with me, and you'll see how it all comes together in a couple slides.

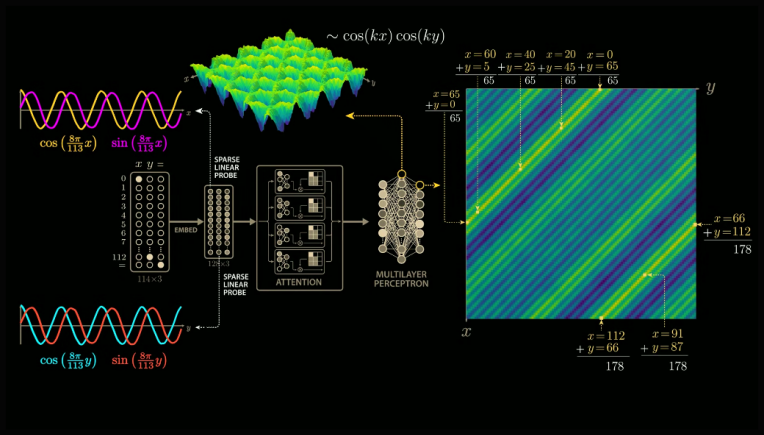
Slide 17 — The diagonal

TRACE 08 // IT LEARNED TO ADD

The diagonal

A single neuron fires for every pair of inputs where **x + y = 65**.

It learned to **add**.



This is where it clicks.

Look at these diagonal stripes. Each stripe is a set of input pairs where the neuron fires maximally.

Look at the numbers along the top stripe: x=0 y=65, x=20 y=45, x=40 y=25, x=60 y=5. What do they have in common? They all add up to 65.

[Pause — let someone figure it out.]

So this neuron fires for every pair of inputs that sum to 65. It learned to detect addition — not by adding, but by geometry.

And the second stripe? Those pairs add up to 178. But 178 mod 113 is 65 — same answer, just wrapped around.

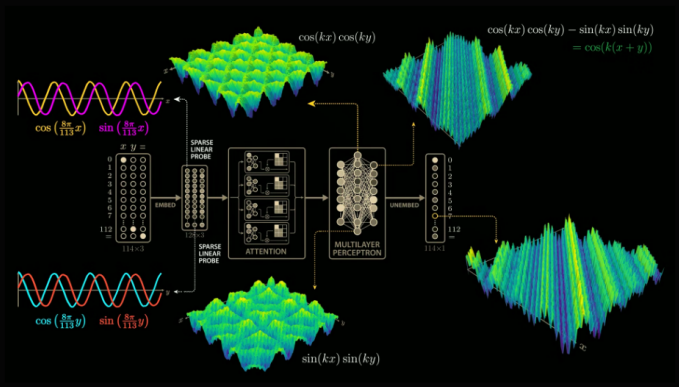
Slide 18 — The trig identity

TRACE 09 // THE TRICK, NAMED

The trig identity

cos(kx)cos(ky) - sin(kx)sin(ky) = cos(k(x+y))

A trigonometric identity converts products of trig functions into a **sum of the inputs**.



Remember that scary equation from the very first slide? This is how it all ties together.

We get our sine and cosine waves, they go through the attention and the perceptron, and come out as those 3D surfaces — and the answer reads out on the right.

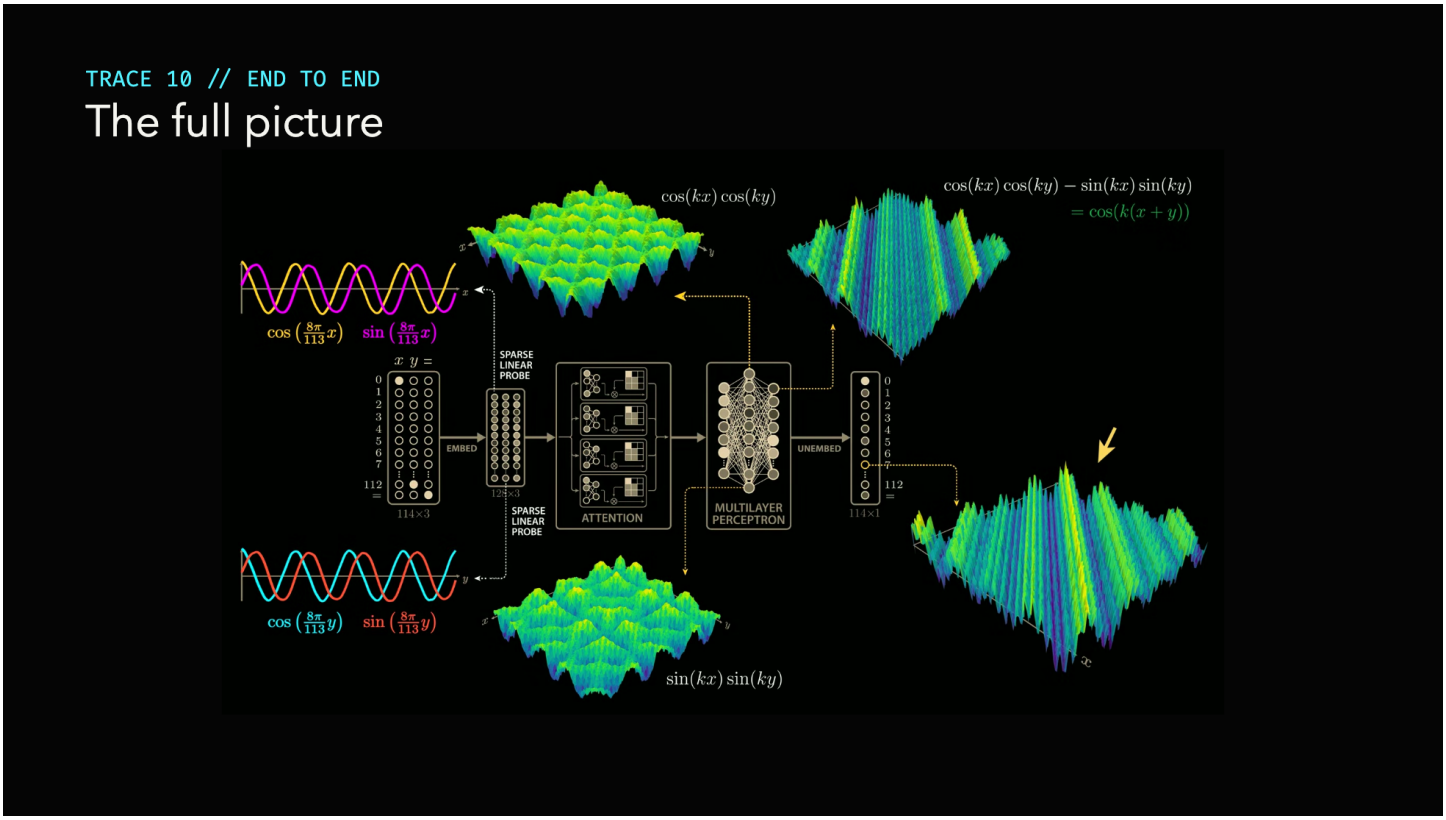
You can see the equation here: cos(kx)cos(ky) - sin(kx)sin(ky) = cos(k(x+y)). All that's to say — you can combine sines and cosines to get addition back out. It's a trick that's been in textbooks for centuries.

Nobody wrote that identity into the model. It emerged on its own, because this representation solves the problem cleanly.

So it didn't really learn addition. It learned geometry.

(beat) ...when's the last time you had to do something like this, huh?

Slide 19 — The full picture



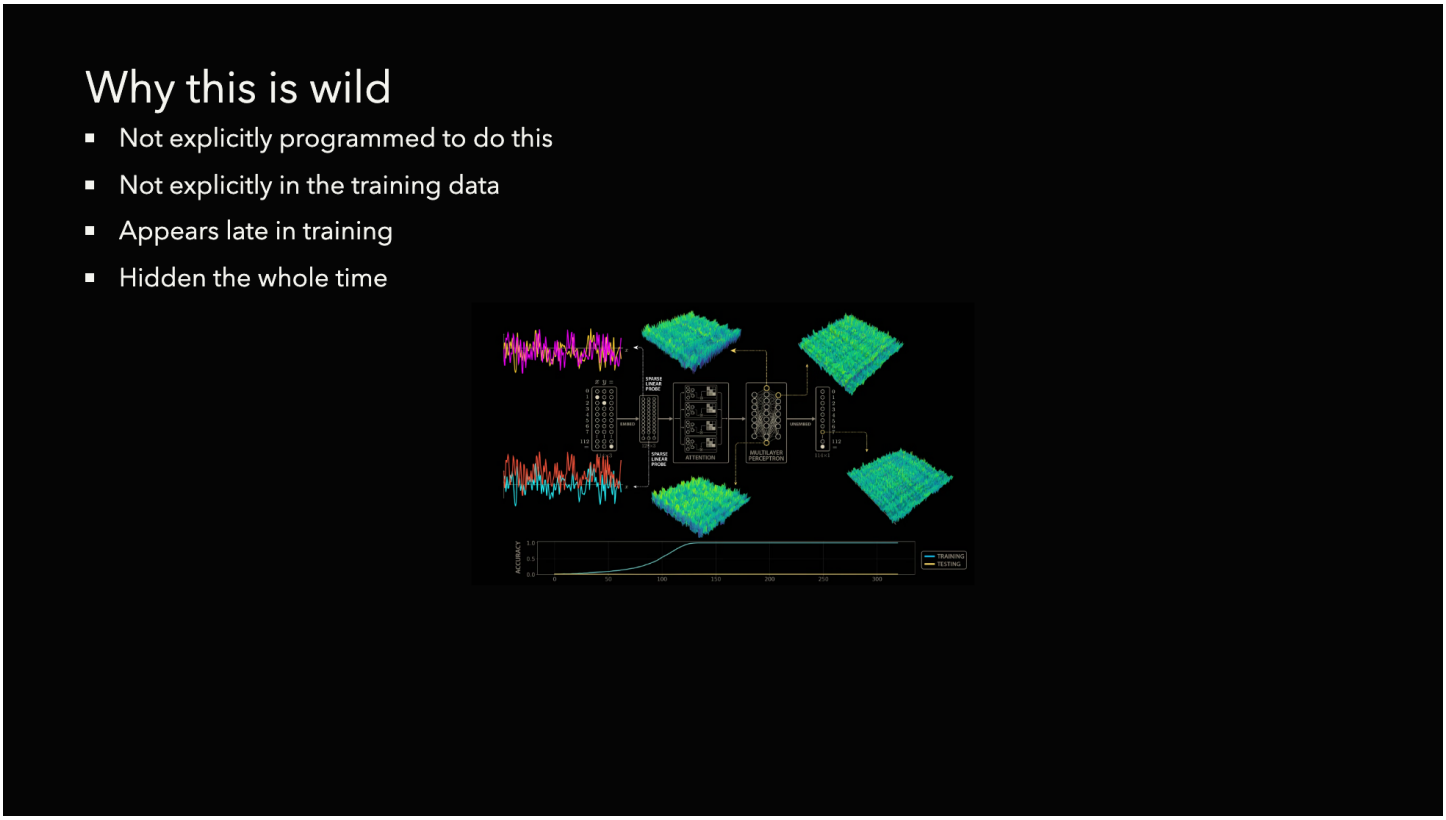
Here's the whole pipeline. Numbers go in on the left, the answer comes out on the right.

Walk it through: the numbers go into the sparse linear probe, which gives us cosine and sine of x and y. That feeds into the attention and the multi-layer perceptron, which gives us those 3D surfaces. And that reads out as an answer — one of those outputs lights up for the right sum, like the 65 line we just looked at.

You don't need to follow every piece. The point is every step has a purpose, and the model built this whole thing from scratch — the representation, the strategy, the math.

It learned a space where the problem becomes easy.

Slide 20 — Why this is wild



So let's step back and think about what just happened.

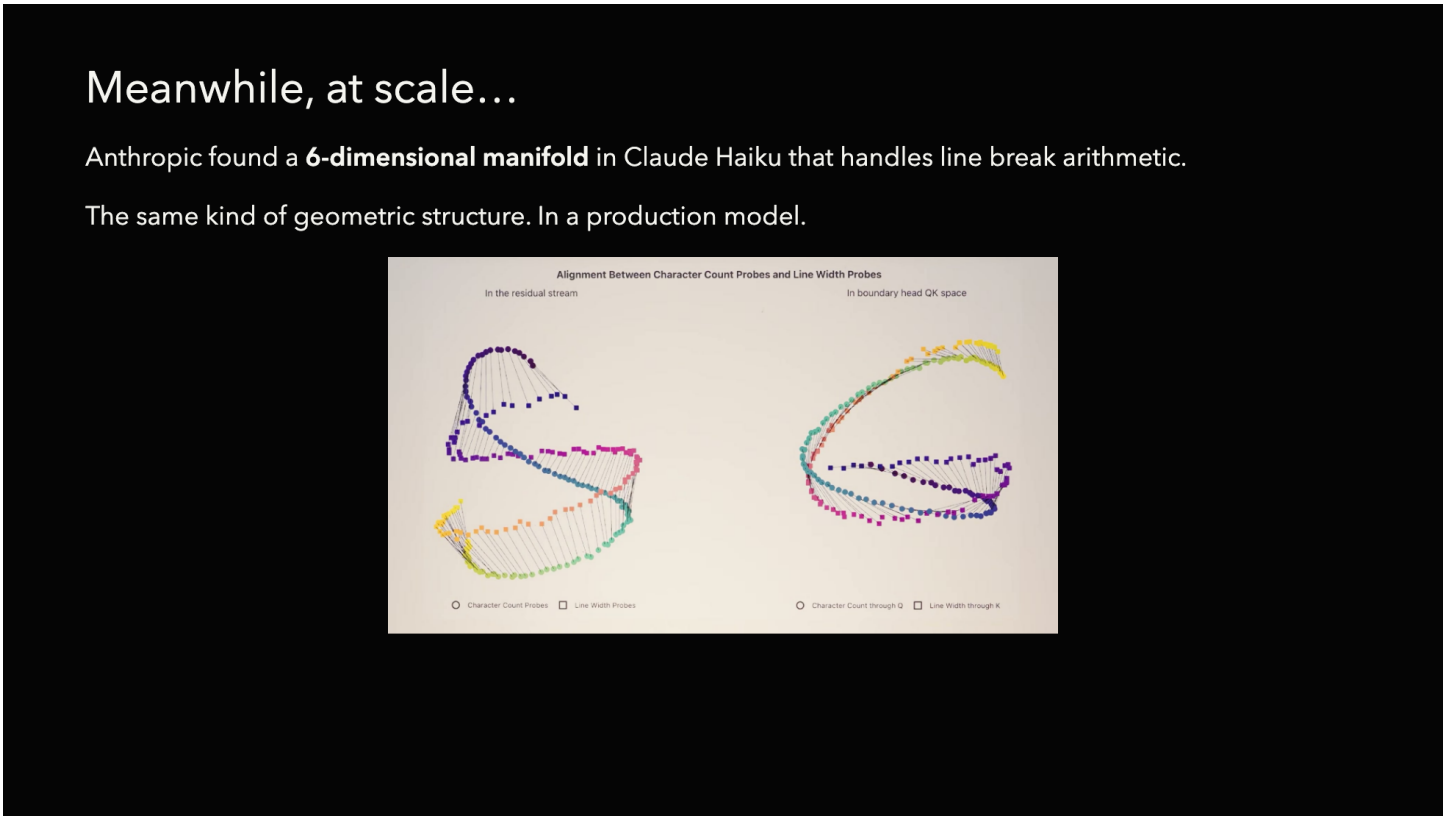
The model was never told about circles, or sine, or cosine. Nobody wrote trigonometry into the code. The training data was just examples like 14 + 87, mod 113.

The model figured out a representation that was an efficient way to solve the problem — and it just happened to be geometric, trig-like.

And it did it really late. For thousands of steps it looked like nothing was happening — most people would have walked away from the training. (The way the story goes, they only caught it because someone left it running way longer than anyone normally would.) The capability was hidden the entire time.

It looked dumb... until it didn't.

Slide 21 — Meanwhile, at scale...



Here's the part that should make you feel a little weird.

Everything I just showed you was a tiny model — a single-layer transformer trained on a toy problem. You might think: okay, cute, but real models are way bigger, way more parameters.

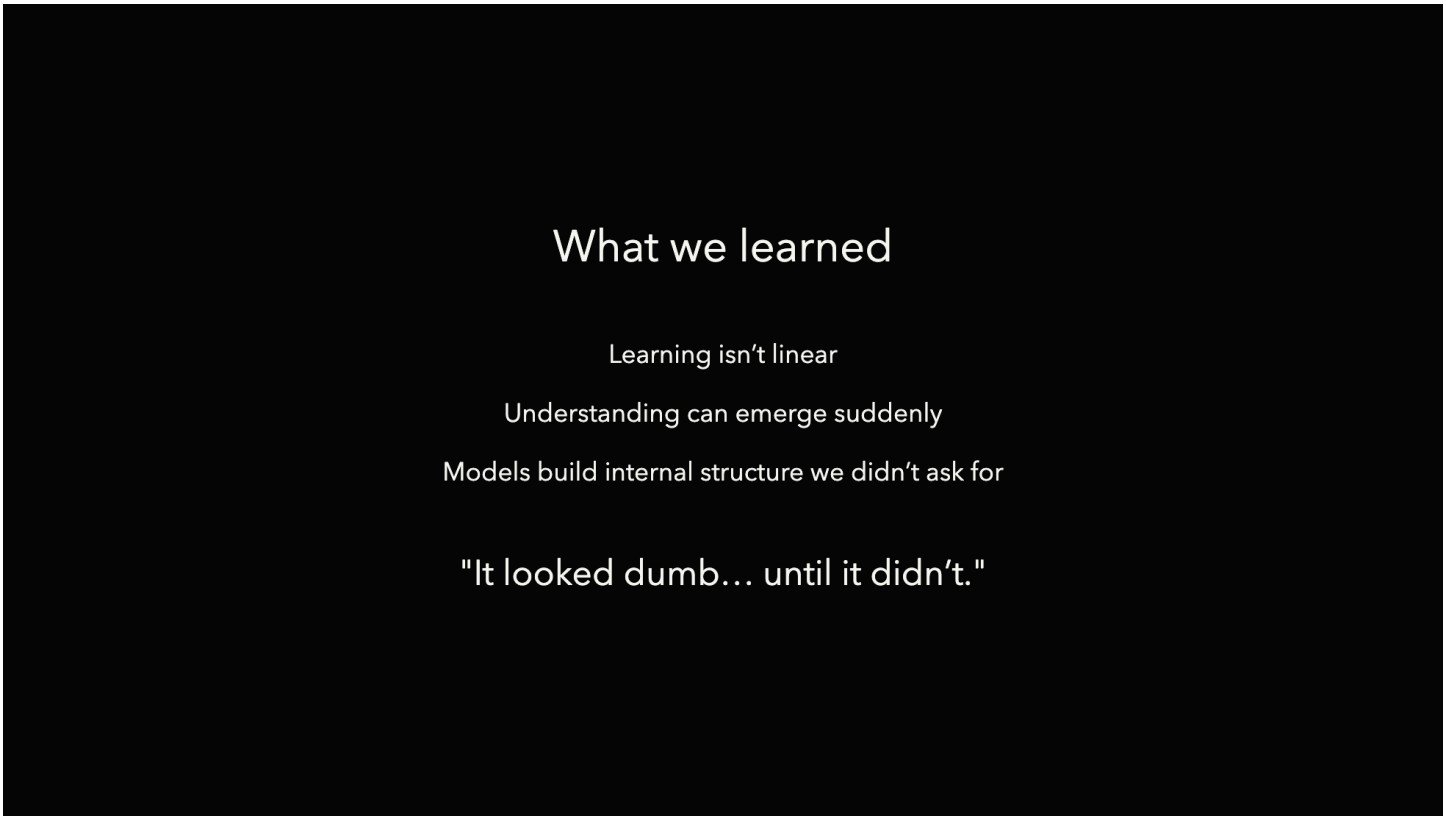
Except Anthropic published a paper where they looked inside Claude Haiku — a real production model — and found the same sort of thing.

When Claude is writing text, it needs to figure out where to put a line break. To do that, it has to count how many characters it's already written on the line. And the way it does that? A six-dimensional geometric manifold — a curved surface in six-dimensional space where character count and line width are positions on a helix. You can see it in this picture: one axis is the character count, the other is how long the line should be.

So even in a model with billions of parameters, we're seeing the same kind of geometric structure — used for something as mundane as line breaks.

If a tiny model rediscovered trig to do addition... what do you think the big models have hiding inside? What else is in there?

Slide 22 — "It looked dumb... until it didn't."



So that's grokking.

Learning isn't linear — a model can look stuck for ages, then snap into a whole new strategy.

Understanding — or something that looks an awful lot like it — can show up all at once, with no warning.

And models build internal structure that nobody asked for. Structure that turns out to be elegant, geometric, and mathematically sophisticated.

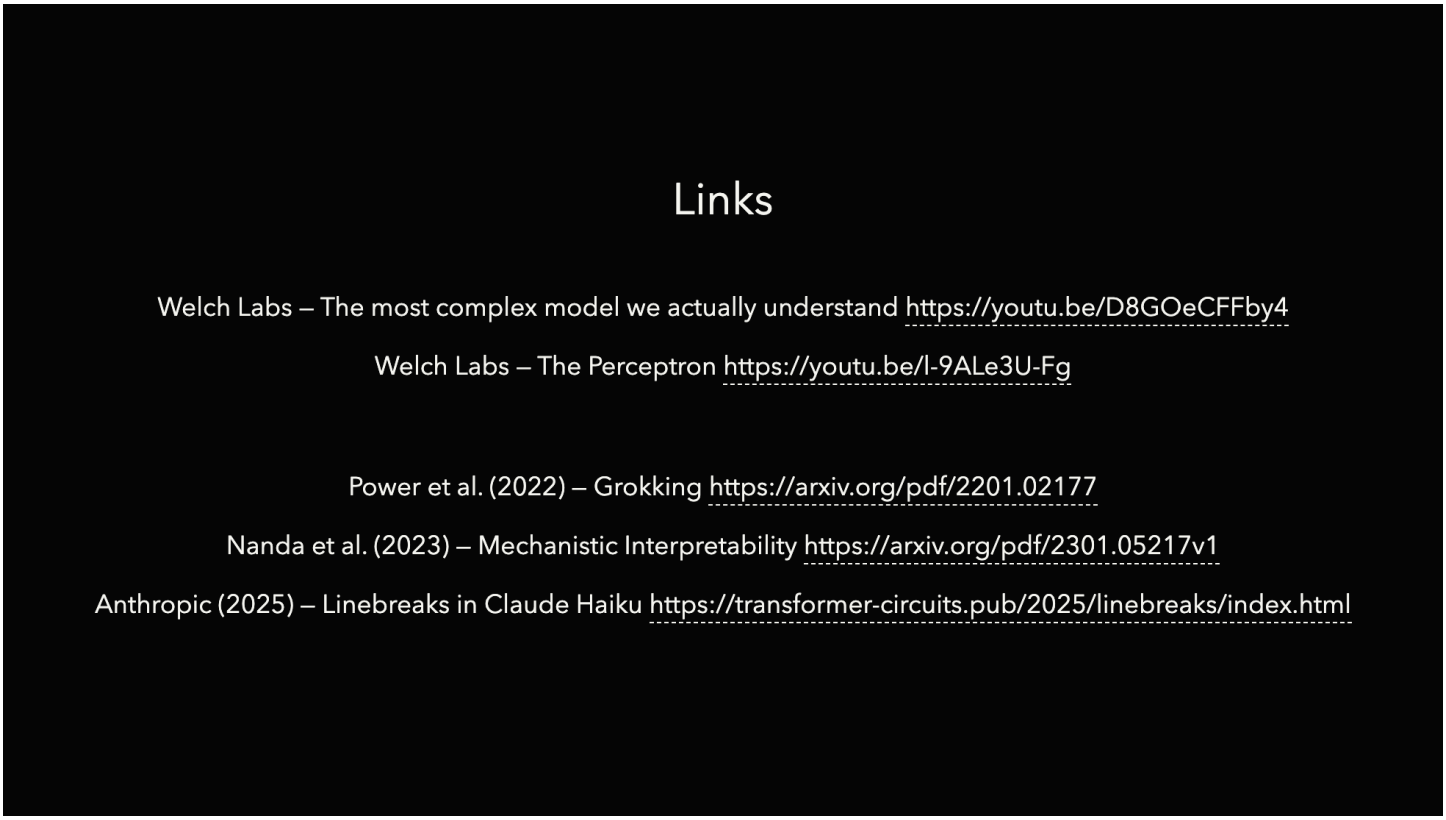
We started tonight by saying AI is just pattern matching. Just statistics. Just optimization. And that's true — technically. But when the optimization discovers trigonometry on its own... maybe 'just' is doing a lot of heavy lifting in that sentence.

Pause.

It looked dumb... until it didn't.

Thank you.

Slide 23 — Links



Leave this up for Q&A. People can take a photo.